

Capstone Data Prep Notes (Music Recommender)

1. Capstone Data Splitting Strategy

What unit is being split?

Songs (rows). Each row is one song with metadata + features.

Which split strategy is most appropriate?

Group-based split by artist (`artist_name`).

Why this split?

For a music recommender, a random row split is misleading because the same artist would show up in train and test. That makes things look better than they really are (the system can “learn the artist”).

So I split **by artist**: - Each artist goes to exactly one split. - All songs from that artist follow. - Goal split sizes: ~80% train / ~10% val / ~10% test (by artist count). - Seeded for reproducibility.

Is a safe split possible?

Yes. This dataset has a clean `artist_name` column and lots of unique artists, so the group split is doable.

2. Required Preprocessing Steps

These are the prep steps I need before I can do similarity search or any ML. This matches what my `main.py` does.

2.1 Drop columns that don't belong in features

I drop: - `Unnamed: 0` (just a row index) - `artist_name` (identity leakage) - `track_name` (basically an ID / shortcut) - `lyrics` (for now — too big to use safely without a real text pipeline)

Why: If I keep IDs (artist/title), the system can “cheat” instead of learning musical similarity.

Assumption: A baseline recommender can still work using audio-ish features + topic scores + genre, even without lyrics.

2.2 Imputation (if anything is missing)

What I do: - Numeric columns → fill with **train median** - Categorical columns (`genre`, `topic`) → fill with **train mode**

Why: Most models/similarity methods don't handle NaNs.

Assumption: Missingness (if it appears) is not hiding some important pattern.

2.3 Scaling numeric features

What I do: Standardize numeric columns using **train mean/std**: $\text{scaled} = (\text{x} - \text{mean_train}) / \text{std_train}$

Why: Features are on different scales (ex: loudness vs danceability). Without scaling, one feature can dominate similarity.

Assumption: Train stats represent the “normal” range of songs I’ll see later.

2.4 Encode categorical features

What I do: One-hot encode **genre** and **topic** using **train-only categories**.

Why: Similarity/search needs numeric vectors. One-hot avoids fake ordering (genre “2” isn’t bigger than genre “1”).

Assumption: Train categories cover most of what I’ll see later.

2.5 Not implemented yet (but planned)

Lyrics features: If I add lyrics later, I’ll use TF-IDF or embeddings (similar idea to the GeeksforGeeks example), but I’ll fit that **only on train** to avoid leakage.

3. Data Leakage Risk Identification

Even without labels, leakage is still a thing. Here are the main ways I could mess this up (and how I avoid it).

Leakage risk 1: Using **artist_name** as a feature

How it could happen: I accidentally keep **artist_name** in the feature set. Then the system just matches artists instead of learning sound/lyrics similarity.

How I prevent it: I drop **artist_name** from features (it’s in **POSSIBLE_EXCLUDES**). I only use it for splitting.

Leakage risk 2: Random row split (artist overlap across splits)

How it could happen: I do a random split by rows. The same artist appears in train and test, so performance looks inflated.

How I prevent it: I split by artist, and I have assertions in **main.py** that confirm **zero artist overlap**.

Leakage risk 3 (future): Fitting TF-IDF/embeddings on the full dataset

How it could happen: If I add lyrics TF-IDF later, it's easy to accidentally fit the vectorizer on train+val+test, which leaks information from the test set into the representation.

How I prevent it: Fit text representations **only on train**, then apply to val/test.

4. What Was Attempted This Week

What I did: - Loaded the dataset and inspected columns. - Implemented the **artist-group split** (train/val/test). - Built a train-only preprocessing pipeline: drop columns, impute, scale, one-hot. - Added checks to ensure splits don't share artists.

What I didn't do (on purpose): - I did not build the full recommender yet (top-N retrieval). This week is about data prep. - I did not use lyrics yet (needs TF-IDF/embeddings and more careful work). - I did not do user-based evaluation because there is no user interaction data.

Constraints: - No user logs → evaluation will be proxy/qualitative for now. - Lyrics are huge/high-cardinality → can't use them naively. - Time/scope → focused on safe prep first.

5. Observations

- The dataset version I used had **no missing values**, but I kept imputation code anyway (defensive + leakage-safe).
- **lyrics** are basically **unique per song**, so they need real text processing (not one-hot).
- Group-splitting by artist makes the genre mix **slightly different** across splits (expected).
- Basic numeric stats (means/stds) across splits looked **pretty similar**, so the split doesn't look wildly biased.

6. Interpretation and Judgment

I think this dataset is in good shape for a **baseline content-based recommender**. The features are reasonable for "song similarity," and the prep pipeline is leakage-safe (fit on train only). The artist-group split is the biggest "realistic evaluation" improvement compared to a random split.

The weakest part is evaluation. Without users (plays/likes/skips), it's hard to say what "good recommendations" means in a measurable way. Also, skipping lyrics is a tradeoff: it keeps things simpler and safer this week, but lyrics might matter a lot for what humans feel is "similar."

7. Forward-Looking Adjustment

Next steps I'd take: - Actually implement the recommender (cosine similarity → top-N nearest songs). - Add lyrics the right way (TF-IDF/embeddings fit on train only). - Pick a proxy evaluation plan (genre/topic agreement + human spot-checking). - Keep an eye on split imbalance (group splits can shift distributions).

8. Mismatch Acknowledgment (If Applicable)

Not a total mismatch, but there's a real limitation: this dataset supports building a content-based recommender, but it doesn't support **standard recommender evaluation** (no users/sessions/feedback).

Even with that limitation, this week was still valuable because I now have a clean, reproducible, leakage-safe split + preprocessing foundation to build on.